Jullian Edrain R. Sevilla - CPE21S4

Sections	Answer
Header File Declaration Section	#include <iostream> using namespace std;
Global Declaration Section	int count = 0;
Class Declaration and Method Definition Section	cpp class Triangle { private: double totalAngle, angleA, angleB, angleC; public: Triangle(double A, double B, double C); void setAngles(double A, double B, double C); const bool validateTriangle(); };
Main Function	cpp int main() { Triangle set1(40, 30, 110); if (set1.validateTriangle()) { cout << "The shape is a valid triangle.\n"; } else { cout << "The shape is NOT a valid triangle.\n"; } return 0; }
Method Definition	cpp Triangle::Triangle(double A, double B, double C) { angleA = A; angleB = B; angleC = C; totalAngle = A + B + C; } void Triangle::setAngles(double A, double B, double C) { angleA = A; angleB = B; angleC = C; totalAngle = A + B + C; } const bool Triangle::validateTriangle() { return (totalAngle <= 180); }

Observations	Comments
The program compiled successfully and displayed the output without errors.	It shows that the syntax and structure of the C++ code were correct.
When the angles added up to 180 or less, the output said “The shape is a valid triangle.”	This confirms that the validation function worked properly.
Changing the angles to exceed 180 made the program print “The shape is NOT a valid triangle.”	The conditional statement behaved as expected, proving the logic was accurate.
The class and methods were easy to understand and reuse.	It helped me see how encapsulation and constructors work in C++

Overall, the activity helped me review basic OOP concepts.	I learned how to define classes, use constructors, and apply logical validation in C++.

Supplementary Activity

main.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int a, b, temp;
    cout << "Enter two numbers: ";
    cin >> a >> b;

    cout << "Before swap: a = " << a << ", b = " << b << endl;

    temp = a;
    a = b;
    b = temp;

    cout << "After swap: a = " << a << ", b = " << b << endl;
    return 0;
}
```

Output

```
Enter two numbers: 67 76
Before swap: a = 67, b = 76
After swap: a = 76, b = 67


=== Code Execution Successful ===
```

main.cpp

```cpp
#include <iostream>
using namespace std;

double kelvinToFahrenheit(double kelvin) {
    return (kelvin - 273.15) * 9/5 + 32;
}

int main() {
    double kelvin;
    cout << "Enter temperature in Kelvin: ";
    cin >> kelvin;

    cout << kelvin << " K = " << kelvinToFahrenheit(kelvin) << " °F" << endl;
    return 0;
}
```

Output

```
Enter temperature in Kelvin: 1
1 K = -457.87 °F


=== Code Execution Successful ===
```

main.cpp

```cpp
#include <iostream>
#include <cmath>
using namespace std;

double distance(double x1, double y1, double x2, double y2) {
    return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}

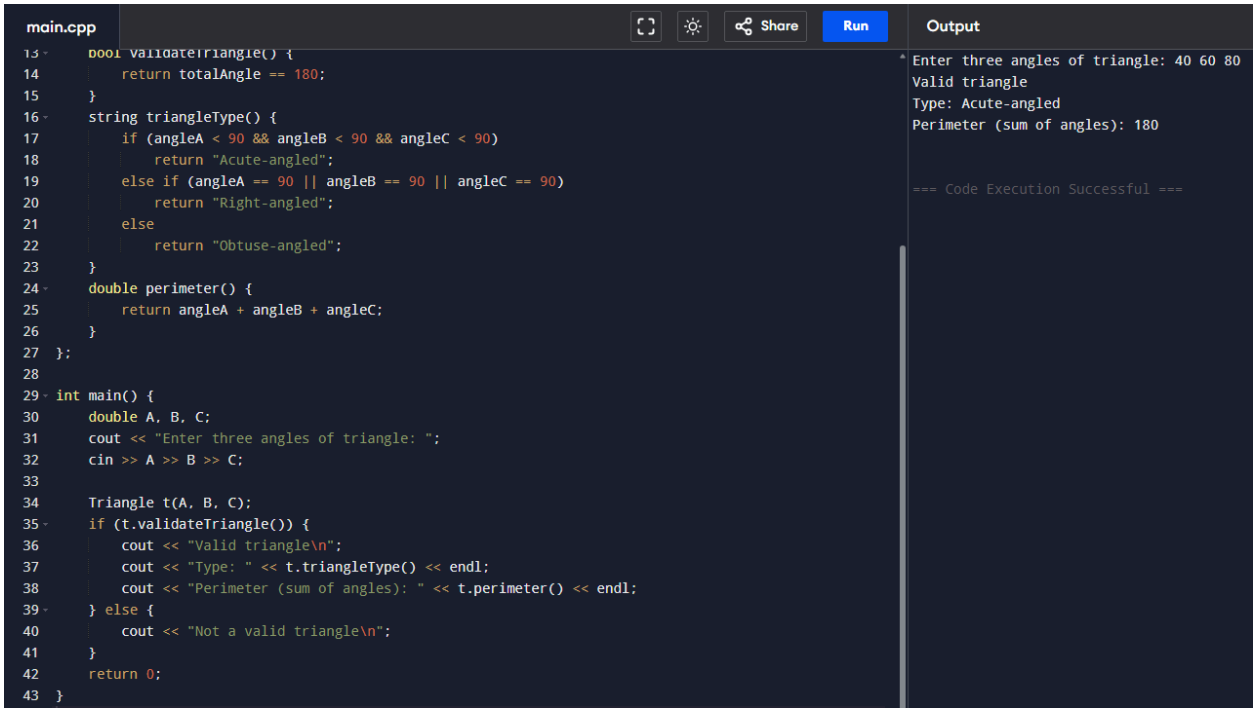
int main() {
    double x1, y1, x2, y2;
    cout << "Enter coordinates (x1 y1): ";
    cin >> x1 >> y1;
    cout << "Enter coordinates (x2 y2): ";
    cin >> x2 >> y2;

    cout << "Distance = " << distance(x1, y1, x2, y2) << endl;
    return 0;
}
```

Output

```
Enter coordinates (x1 y1): 6 7
Enter coordinates (x2 y2): 6 5
Distance = 2


=== Code Execution Successful ===
```



Conclusion
Summary of Lessons Learned:

In this activity, I learned how to apply basic C++ programming concepts, including functions, classes, and object-oriented programming. I understood how to structure code properly using header declarations, global variables, and method definitions. The exercises helped me strengthen my logic and problem-solving skills.

Analysis of the Procedure:

The step-by-step procedure made it easier to follow the C++ structure. Writing the class and method definitions separately improved my understanding of encapsulation and reusability. Running the programs in Programiz allowed me to test and debug efficiently.

Analysis of the Supplementary Activity:

The supplementary tasks challenged me to think critically and apply formulas in real-world scenarios. I practiced creating functions to swap numbers, convert temperatures, compute distances, and validate triangles. Each problem reinforced how C++ can be used to solve mathematical and logical problems.

Concluding Statement / Feedback:

I think I did well in this activity because I was able to run all programs successfully and understand their outputs. My area for improvement is learning more advanced C++ syntax and optimizing my code for efficiency. Overall, this activity helped me gain confidence in writing structured and object-oriented programs.